

명성

+821031800830 @wsadqe@naver.com

백엔드 개발을 중심으로 AI/LLM, 데이터 처리, 플랫폼 고도화 프로젝트를 수행해온 개발자입니다.

신규 서비스 기획부터 PoC, API 서버 설계·개발, 외부 시스템 연동, 상용화 및 운영 안정화까지 전 과정을 경험했으며, 실제 서비스에 적용 가능한 구조를 설계하고 구현하는 데 강점이 있습니다.

RAG 챗봇, Prompt Security, NFT/블록체인 서비스, 사내 운영 플랫폼 등 다양한 도메인에서 문제 해결 중심의 개발 경험을 보유하고 있습니다.

경력 7년 7개월



디지캡

2018.11 - 재직중 (7년 7개월) | 정규직 | 서버 개발자 | 선임

스마트글라스를 이용한 화상회의와 업무관리 서비스 개발

2025.01 - 2025.12 | 백엔드 API서버 개발, 파일, 작업로그 서버 담당 개발 | 선임

프로젝트 개요: 공장, 지하 등 통신 인프라가 취약하고 두 손을 자유롭게 사용할 수 없는 현장 실무자들과 오피스 지원 인력 간의 실시간 소통 부재는 업무 효율 저하와 안전사고 리스크를 야기했습니다. 이를 해결하기 위해 현장의 시야를 실시간으로 공유하고 오피스와 긴밀히 협업할 수 있는 무중단 화상회의 백엔드 인프라 구축이 핵심 과제였습니다.

화상회의 서버 및 API 서버 개발

특정 스마트글라스 하드웨어에 종속적인 임베디드 개발 방식 대신, 시장에 출시된 다양한 스마트글라스 안드로이드 OS 환경에 유연하게 마이그레이션이 가능한 범용 애플리케이션 연동형 API 아키텍처를 채택하여 플랫폼의 확장성을 극대화했습니다.

스마트글라스의 제한된 하드웨어 리소스와 미약한 통신 환경(네트워크 대역폭 제한)을 고려하여, 어플리케이션 자체는 안드로이드 네이티브 언어를 이용해 최적화 개발을 했고, 저지연(Low-Latency) 실시간 미디어 스트리밍이 가능한 비동기 REST API 및 최적화된 소켓 통신을 제공하는 오픈소스를 이용해 화상회의를 구현했습니다.

네트워크 음영지대에서 발생할 수 있는 화상회의 화질저하나 연결 끊김 현상을 방지하기 위해, 스마트글라스에서 현재 네트워크 상태를 확인하고, 그에 따라서 화상회의의 화질과 대역폭을 조절하여 화상회의의 지속성을 유지했습니다.

파일서버 및 체크리스트 관리 서버 개발

현장 근무자들이 노트나 태블릿을 들고 체크리스트를 확인하는 방식은 기기의 고장, 낙하 사고를 유발할 수 있습니다.

스마트 글라스 화면에서 업무 체크리스트 나 해야할 일을 실시간으로 서빙하면 두손이 자유로운 상황에서 더 안전하게 현장 근무를 이어나갈 수 있습니다.

단일 DB내에 미디어 파일에 대한 In/Out, 체크리스트 결과에 대한 In/Out 이 섞이면 테이블이 분리되어있어도 불필요한 병목현상이 발생할 수 있으므로, 일반적인 API를 사용하는 DB와 복잡한 체크리스트를 저장하는 DB로 분리하여 데이터들을 관리했습니다. 스마트 글라스는 일반적인 단말기기보다 하드웨어적으로 사양이 부족하기때매 그에 맞게 API 결과 데이터를 경량화하고 불필요한 A

PI 호출을 최소화할 수 있도록 했습니다.

네트워크 이슈때문에 스마트클래스에서 촬영한 영상/미디어 파일의 업로드가 느려질 수 있으므로, 청크 기반의 업로드를 구현하여, 중간중간 네트워크의 On/Off가 반복되더라도 중지된 부분부터 결과파일을 업로드 할 수 있는 방식을 제공했습니다.

아쉽게도 스마트 글라스 서비스의 실제 상용서비스 오픈으로까진 이어지지 않았지만, 자체적인 개발 완료 후 임원진들 발표 시 좋은 평가를 받았고, 고도화에 대한 진행여부까지 얘기가 나왔었습니다.

기존 사내 카페 키오스크 사용방식의 고도화 주도

2024.07 - 2024.10 | API 개발, 백엔드 개발자 | 대리

프로젝트 개요: 기존 사내 카페 키오스크는 신원 인증 절차가 사원카드(RFID) 태깅 방식으로 운영되었습니다. 그런데 카드 분실/미소지 시 서비스 이용이 불가능하고, 다른 사람의 카드로 이용할 수 있다는 단점이 있었습니다. 또한 출력된 영수증을 사용자가 직접 카페로 제출해야 주문이 확정되었습니다

이로 인해

(1) 타인 명의 도용 및 주문 오입력에 대한 검증 불가

(2) 영수증을 출력해 카페에 직접 제출하는 과정에서 병목 현상이 발생했습니다.

서비스 신뢰도를 높이고 결제-주문 딜레이를 최소화하기 위해 자동화된 인증 체계 도입을 기획했습니다.

인증 방식 선택: 별도의 소지품 없이 신속하고 정확한 신원 검증이 가능한 안면인식 하드웨어 디바이스 연동 방식을 채택했습니다.

데이터 스키마 설계: 인식 기기의 식별 데이터와 사내 임직원 데이터를 안전하게 연결하기 위해, 전 사주의 인증 사진 데이터를 기반으로 고유 ID(사번) 및 초기 자격 증명(비밀번호) 매핑 테이블 구조를 설계하여 데이터 무결성을 확보하고자 했습니다.

안면인식 기기와 키오스크 간의 실시간 연동 시 네트워크 레이턴시(Latency)로 인해 결제 전환 속도가 저하될 수 있는 트레이드오프가 있었습니다. 이는 하드웨어 기기에 기본적인 안면인식 정보를 넣어두고, 오프라인 혹은 긴 레이턴시가 걸리더라도 우선적으로 캐싱되어있는 안면인식 정보로 넘어가고, 그 이후 네트워크 속도가 정상화 되었을 때 기록 정보를 보내는 방식으로 진행했습니다.

이렇게 개발된 안면인식 + 키오스크 방식은 현재까지도 성공적으로 사용되고 있고, 안정성 99%이상을 유지하며 오류로 문의가 들어오는 경우는 월 1회 이하입니다.

연합뉴스 아카이브 서비스 프론트엔드 개발 및 기획 과 운영 오픈 지원

2021.03 - 2022.02 | 프론트엔드 개발, 기획 지원, 서비스 운영 오픈 지원 | 대리

프로젝트 개요: 기존 연합뉴스에서 지원하던 뉴스기사 이미지, 영상, 음성, 인물 정보 판매 사이트에 대한 대규모 리뉴얼.

개발 초기 기획 지원 (회의 참석을 통한 다양한 사람들의 커뮤니케이션 허브역할 과 디테일을 기억할 수 있게 회의록 작성)

사이트 개발에 대한 착수 직후엔 클라이언트, 기획자, 개발자 등 많은 이해관계자들간의 대화가 가장 많이 이루어 집니다. 기획 요구 사항과 기술적 구현 가능성의 차이, 불분명한 의도 전달 등 이러한 것들은 설계의 잦은 변화나 개발시간 지연 이라는 위험을 야기합니다.

아직 주니어의 입장에서 실제 결정권을 갖거나 실무에 적용할 내용을 직접적으로 다룰 순 없었으나, 오히려 회의나 사담을 나누면서 사소하게 넘어갈 수 있는 디자인에 대한 요청사항, 기능에 대한 검토 요청 사항을 모두 작성하고 메인 PM과 공유했습니다.

이는 초반엔 이미 회의때 다 나온 얘기라며 불필요한 행동이라는 소리를 들었습니다. 그러나 시간이 조금씩 지나며 가볍게 넘어갈 수 있는 항목들도 세세하게 기억한다는 좋은 평가를 클라이언트로부터 받게 하는 계기가 되었습니다.

소셜 로그인을 통한 통합 인증 페이지 개발

일반 사용자들의 연합뉴스 접근성을 높이고자, 기존 아이디/패스워드 를 이용한 회원 관리에 소셜(네이버, 카카오) 로그인 기능을 추가했습니다.

보안과 확장성을 위해 각 소셜 플랫폼에서 서비스하는 OAuth 2.0 규격을 준수하되, 내부에서 사용하고 있는 회원체계와의 매핑구조를 확인하고 회원정보의 무결성을 확보했습니다.

또한 단순 소셜 로그인 기능과 인증에 대한 보안 확인이 아닌 외부 툴을 이용한 XSS 나 CSRF 보안 취약점 검토 등 프론트엔드에서 필요로 하는 보안로직을 구현했습니다.

소셜 플랫폼 별로 다양하게 사용되는 API 응답 구조와 인증 프로세스를 통합 처리하기 위해 인증 로직을 공통 모듈화하고 보안 점검 시 발견된 취약항목 (민감데이터 노출 등) 을 반영했습니다.

이는 네이버와 카카오의 자체 검사를 통과해 소셜로그인 기능을 상용 오픈할 수 있었고, 보안 전문 툴의 점검을 통과하여 문제없이 서비스를 런칭했습니다.

이후 클라이언트에게 코드레벨 인수인계를 교육하고 자체적으로 수정/개발 을 할 수 있도록 도와주었습니다.

CQMS 서비스 고도화 개선 (Redis + RDB -> Elasticsearch)

2019.03 - 2020.03 | 백엔드 개발자 | 사원

프로젝트 개요: 기존 CQMS 시스템을 개발-사용 하다가 느끼게 된 초기 구조의 한계. 이런 한계를 해결하고, 안정적인 서비스를 위해 서빙하는 데이터베이스와 구조를 개선함

백엔드 DB 마이그레이션 (서비스 데이터 저장소인 Redis 와 RDB를 Elasticsearch로 이관)

기존 CQMS 는 RDB와 Redis 로 데이터를 저장했습니다. 이러한 구조는 서비스를 위한 복잡한 데이터가 필요할 때, 미리 모든 경우에 대해서 가공하고 단순한 Key-Value 로 저장하여 사용해야 했는데, 이러한 불필요한 과정을 줄이고, 대용량 데이터를 안정적으로 서빙하기 위한 아키텍처의 변경이 필요했습니다.

한번 데이터가 INSERT 가 된 이후에는 Update 나 Delete 가 거의 발생하지 않았고, 이는 Elasticsearch 와 데이터구조가 적합하다고 생각했습니다. 또한 기존 RDB에 비해 Elasticsearch는 분산저장을 통한 Scale-Out으로 데이터가 많아지더라도 인프라에 부담을 적게 줄 수 있었습니다.

초기 Elasticsearch 도입 시 러닝커브의 이슈로, Indexing 과 Replica 에 대한 이해가 없었습니다. 이를 해결하기 위해 Elasticsearch에 대한 학습을 진행하였고, 대량 적재 시 Indexing interval 조절과 여러대의 VM을 통한 Replica 설정으로 노드장애로 인한 데이터 유실과 서비스 장애를 해결했습니다.

메모리 부족과 Procedure 수행 속도 문제로 2시간씩 걸리던 데이터 가공 -> API 서버 동작에 대해서 시간을 10분단위까지 줄일 수 있었고, 사용자가 필요로하는 데이터에 대한 제공을 손 쉽게 서비스 할 수 있었습니다.

백엔드 API 서버 리팩토링

초기 API 서버 설계 시 역량 부족으로 제대로된 아키텍처 설계가 없이 구성된 API 서버는 개발이 지속될수록 하나의 파일에 라우팅, 비즈니스로직, DB 접근이 혼재하는 스파게티 코드가 되었는데, 이는 기능추가와 버그수정시 사이드이펙트 파악이 어려웠습니다. 또한 새로운 개발자가 온보딩 하는데에 시간이 많이 소요되는 상황이 발생했습니다.

파일별로 역할에 따라서 분리하는 Layered 아키텍처를 도입하였고, 컨트롤러와 서비스, 모델 등으로 계층을 분리하였습니다.

파일이 늘어나고, 복잡해지는것처럼 보였으나 오히려 사이드이펙트가 50%이상 줄어들었습니다. 또한 callback으로 구현되어있던 소스들을 Async/Await 으로 통일해 코드들의 진행흐름을 일관성있게 수정했습니다.

이러한 결과 기능 개발을 위해 투입되는 시간이나 야근율이 기존대비 30% 가량 줄어드는 결과를 얻었습니다.

SKB 내 CQMS(Contents Quality Management System) 개발

2018.11 - 2019.03 | 풀스택 개발자, 기획자 | 사원

프로젝트 개요: SKB에서 서비스하는 다양한 오디오/비디오 콘텐츠들의 인코딩 포맷을 결정하고, 소비자들의 실제 이용량을 파악하는 서비스. 일일 100만건 이상의 소비자 로그데이터와, 서비스하는 20만건의 콘텐츠 데이터를 시각화 하고 확인하기 위한 서비스

백엔드 담당 (하둡 원천 로그 데이터의 MySQL 자동 적재 및 API 파이프라인 구축)

SKB에서는 하둡을 통해서 일간 100만건 이상의 소비자 로그데이터를 지속적으로 쌓고 있었습니다. 이는 만들고자 하는 실시간 API 성 서비스에는 적합하지 않았습니다. 때문에 특정 타이밍에 쌓여진 로그데이터를 사용자가 조회할 수 있도록 관계형 DB로 이관하고, 서비스 가능한 형태로 가공해야 했습니다.

이 때 관계형 DB는 MySQL 과 Oracle DB 를 고민했으나, 라이선스의 이슈로 MySQL 을 선택했습니다.

하둡 to MySQL 데이터 파이프라인에는 Apache Sqoop 라는 ETL을 고려했으나, 관리포인트 증가와 유지보수 비용에 대한 이유로 단순화 하였습니다. "CSV파일" 생성 -> Shell Script 실행 -> MySQL 의 "LOAD DATA INFILE" 명령어를 사용해 고속으로 Bulk 하는 방향으로 결정했습니다.

적재된 데이터는 서버인 NodeJS 를 이용해 가공할 경우 OOM 이슈가 발생할 수 있어, 임시테이블에 적재하고 MySQL 의 Procedure 를 이용해 가공 > 적재하는 과정을 거치도록 설계했습니다.

파일적재 기반의 방식은 구현이 직관적이고 빠르지만, (1) 파일의 스토리지 용량차지, (2) 대량 데이터 Insert 시 DB Lock 의 이슈가 발생할 수 있습니다. 이에 Linux의 Cron Batch 를 이용해서 사용자의 유입이 없는 새벽 4시부터 수행되도록 했고, 1주일간 원천파일 보관/6개월간 DB데이터 보관 등 데이터 저장 정책을 수립했습니다.

하둡으로 추출된 CSV 파일의 생성 및 Transfer -> DB 적재 -> Procedure 수행을 통한 데이터 가공 -> API 서버 동작 으로 진행되는 전 과정을 자동화하고, 많은 양의 하둡 데이터를 장애나 지연을 최소화하여 사용자에게 서비스할 수 있었습니다.

프론트엔드 및 기획 담당 (화면 기획회의의 참석, 사용자 개개인의 개인화 컴포넌트 화면 개발, 다양한 차트 구성)

획일화 된 단순 표로만 방대한 데이터를 제공하면, 사용자가 정보를 해석하기 어렵고 피로도가 증가합니다. 또한 사용자별로 같은 데이터를 해석하고 사용하는 방식이 다르기때문에 다양한 차트를 제공하되, 화면을 구성하는 컴포넌트를 자유롭게 배치할 수 있어야 했습니다.

이를 위해 Charts.js 와 HighCharts 를 고민했으나, 복잡한 데이터를 다양한 차트를 제공할 수 있고, 브라우저의 호환성을 최대한 지원하는 HighCharts로 결정했습니다.

하나의 화면에서 다양한 차트를 그릴 때 데이터에 대한 초기 렌더링 속도가 느려지는 이슈가 발생합니다. 이를 개선하기 위해 화면상에 차트를 그리기위한 최소 데이터를 먼저 API를 호출하여 제공하고, 각각의 차트 컴포넌트들을 비동기로 호출하여 화면이 그려지는 체감 시간을 최소화 했습니다.

이러한 웹 서비스는 사용자들의 호응을 이끌어냈고, 2018년 2019년 SKB 내 성공적인 프로젝트로 상을 수상했습니다.

학력



강원대학교

2010.03 - 2019.02 | 졸업 | 컴퓨터정보통신공학과